

Gobernanza y Orquestación

De n8n a Python

II-5 Implementación y Pruebas · II-6 Operación y Despliegue

MVP-A VERBEX COMPOSITES · COIIAOC 2026

PRDA · Sistema

repo verbex-agent

II-5 + II-6 · 2,5 h

Implementación y Pruebas

Modularidad y cohesión

Trade-off monolítico / modular

X MONOLITO

- 26 nodos en un workflow
- cambiar el prompt toca todo
- probar una regla = correr todo

✓ MODULAR

- llm · rules · persistence · notify
- cada módulo, una responsabilidad
- rules.py se prueba sin red (E5)

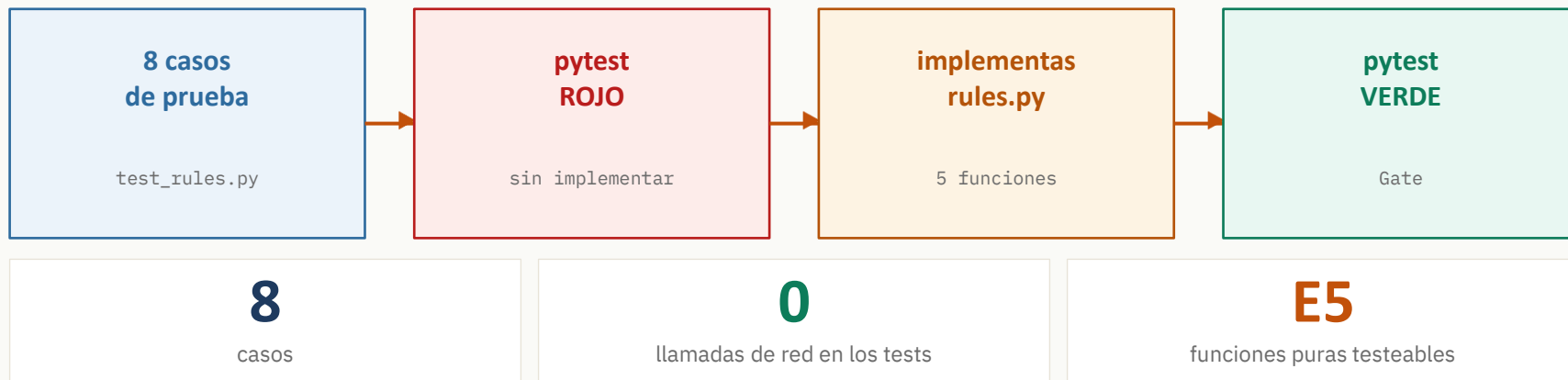
Taxonomía de fallos

TIPO	QUÉ ES	MITIGACIÓN EN PYTHON
Transitorio	el reintento lo resuelve	reintento + log en verbex.log
Permanente	requiere intervención humana	excepción capturada → Errores.csv
Silencioso	el más peligroso: pasa sin avisar	alertas[] + estado visible en dashboard

Bulkhead: módulos aislados evitan que un fallo en *notify* tumbe la extracción

Circuit-breaker: el *parser* anti-alucinación corta antes de persistir basura

Tests como especificación



La especificación no es SÓLO un documento, alimenta n script de ejecutable
Los 8 tests = 8 casos validados en el nodo Code de n8n

El pipeline en código



main.py es el grafo del workflow en código: las conexiones de n8n se vuelven llamadas secuenciales y condicionales.

Estructura de ficheros

rules.py

5 funciones puras

Reciben un dict po, devuelven un dict po. Sin LLM, sin ficheros. Orden: R04+R08 → R07 → R01+R02 → R05+R06 → R03.

pytest

el Gate

8 casos en rojo en el esqueleto. Cuando rules.py está completo, pasan a verde: ese es el criterio.

main.py

el orquestador

Encadena llm → reglas → R09 → fan-out. Los TODO numerados siguen el grafo del workflow de II-4.

telegram_bot.py

ya dado

El trigger que recibe POs y llama a main.process(). Lo levantas con python telegram_bot.py.

R01–R09 como funciones

FUNCIÓN	REGLA	DECIDE	TEST
r04_r08_validar_lineas	R04·R08	PN fuera de catálogo o cantidad inválida → ERROR	test_r04 · test_r08
r07_validar_cliente	R07	cliente no en ERP → PARCIAL + flag	test_r07
r01_r02_prioridad	R01·R02	AOG o fecha ≤ 7 días → prioridad	test_r01 · test_r02
r05_r06_doc_requerida	R05·R06	CoC / EASA Form 1 → doc_requerida	test_r05_r06
r03_confianza	R03	confianza < 0,80 → PARCIAL	test_r03

R09 (anti-duplicado) vive en *persistence.es_duplicado()*, no en *rules.py*: necesita leer el CSV, así que no es una función pura.

Operación y Despliegue

Los tres pilares

Logs

qué pasó

observability.py escribe eventos en *logs/verbex.log*
La *stdlib logging*, no un servicio externo

Métricas

con qué frecuencia

El dashboard agrega sobre los CSV:
POs, tasa de éxito, AOGs, errores 7 días.

Trazas

por qué falló

Errores.csv guarda tipo, origen y *error_raw*

Los ERROR de negocio (PO rechazada por una regla) viven en *Pedidos.csv*
los fallos técnicos (SMTP, API) en *Errores.csv*

Dashboard, deploy y entrega

Dashboard CSV

`streamlit run dashboard/app.py`

Read-only sobre data/*.csv. 5 KPIs, filtros y tabla de últimas POs. Sin credenciales, sin Sheets.

Deploy · 2 procesos

`start_verbex.bat`

Arranca bot + dashboard con doble clic. Task Scheduler lo automatiza al iniciar sesión en Windows.

Gobernanza

`checklist 48 ítems · AS9100`

Seguridad, costes, retention 7 años, trazabilidad. Se repasa antes de producción real.

Proyecto propio

`Tu reto`

Adapta el patrón a otro dominio: *rules.py* propio, tests en verde, **PERSONA/SKILL** nuevos.

Contrato IORE

INPUTS

- Repo verbex-agent (esqueleto)
- Python 3.10+ · pytest
- ANTHROPIC_API_KEY · TELEGRAM_BOT_TOKEN

OUTPUTS

- rules.py + main.py implementados
- pytest 8/8 verde
- dashboard en localhost:8501
- deploy con doble clic (bot + dashboard)

REGLAS

- El LLM extrae, las reglas deciden (E5)
- rules.py = funciones puras
- efectos en verbex/, orquestados por main.py
- fallos técnicos → Errores.csv + log

EXCEPCIONES

- PO no-pertinente → ERROR sin ejecutar reglas
- fallo técnico → Errores.csv + mensaje al chat
- duplicado (R09) → avisa y detiene

Criterio de cumplimiento

Gate ✓

pytest 8/8 verde · el agente procesa una PO end-to-end · el dashboard muestra POs y KPIs desde CSV · start_verbex.bat arranca bot + dashboard.

→ 6ª sesión · CrewAI

- Sistemas multiagente con roles
- Crew de 3 agentes · NCR · 8D · CAPA
- Comparativa n8n vs Python vs CrewAI

Idea clave: «un agente resuelve; varios colaboran»

En términos transferibles

Concepto abstracto

sistema operable

Observabilidad + recuperación de fallos + rastro auditable. No basta con que funcione en demo.

En VERBEX · II-6

agente de POs

logging + dashboard + Errores.csv + gobernanza AS9100 (retention 7 años).

En otro dominio

farma · FDA 21 CFR Part 11

Alertas de planta: audit trail, integridad de datos, trazabilidad. Mismos requisitos.

Pregunta de dominio: en tu proyecto, ¿qué dejaría rastro auditable y cómo demostrarías que el sistema se recupera de un fallo?

¿Tienes un agente o un sistema operable?

Con tests, observabilidad y deploy: un SISTEMA